

PROJECT CODE WALKTHROUGH

FinPulse

A file-by-file explanation of the codebase

Financial News Sentiment Pipeline - ingestion, sentiment, storage, API, dashboard

How the project fits together

FinPulse is a decoupled pipeline. Headlines come from a source (simulated feed or NewsAPI), get scored for sentiment and tagged to a ticker, and are stored in SQLite. An aggregation layer joins that sentiment against real prices on a shared UTC timeline. Two independent layers then serve the same store: a FastAPI REST API and a Dash/Plotly dashboard.

Flow: source -> sentiment + ticker tagging -> SQLite -> aggregation/alignment -> API & dashboard

Stages communicate only through a queue (live streaming) and the SQLite store, so each stage can be swapped independently. The pages that follow document every file: what it is responsible for, what it imports, the functions it defines, its key calls, and a short summary.

finpulse/ingestion/news_simulator.py

Responsible for

Simulated news source - a fixed list of fake finance headlines for offline/demo runs.

External libraries / modules imported

json (imported, currently unused)

Imported from the project

(none)

Functions / key definitions

headlines [list]

20 sample headlines, each annotated with its expected ticker + sentiment.

get_headlines()

Returns the headlines list.

Key calls / interactions

- (none - pure data + a getter)

Summary

The offline news feed. A stand-in for a live source, exposing the same role as news_api but with no network or API key. Consumed by runner.py and batch_test.py.

finpulse/ingestion/news_api.py

Responsible for

Real news source - fetches live finance headlines from NewsAPI (creativity: entity disambiguation).

External libraries / modules imported

os, requests, dotenv.load_dotenv

Imported from the project

(none)

Functions / key definitions

API_KEY / URL / FINANCE_DOMAINS

Module-level config; key read from .env; finance-only source list.

fetch_headlines(query, limit=20)

Calls NewsAPI /everything with qInTitle + finance domains + relevancy sort; returns a list of dicts {text, source, timestamp, url}.

Key calls / interactions

- requests.get(URL, params=...)
- resp.raise_for_status()
- resp.json()

Summary

Real headline ingestion. qInTitle + finance-only domains is the entity-disambiguation feature (so 'Apple' = the company). Returns the SAME dict shape as the simulator, so nothing downstream changes. Used by fetch_news.py.

finpulse/ingestion/price_feed.py

Responsible for

Price ingestion exploration script - pulls daily OHLC for AAPL and prints its structure.

External libraries / modules imported

yfinance

Imported from the project

(none)

Functions / key definitions

(top-level script, no functions)

Downloads 5 days of AAPL daily data and prints the DataFrame, its columns (a MultiIndex) and its datetime index.

Key calls / interactions

- yf.set_tz_cache_location(...)
- yf.download('AAPL', ...)
- print(...)

Summary

A Milestone-1 exploration script demonstrating how yfinance returns price data. The production price logic now lives in aggregator.py (load_price / load_ohlc); this file is a learning artifact.

finpulse/sentiment/analyzer.py

Responsible for

Sentiment scoring - VADER tuned with a custom finance lexicon.

External libraries / modules imported

vaderSentiment.SentimentIntensityAnalyzer

Imported from the project

(none)

Functions / key definitions

analyzer [object]

A reused SentimentIntensityAnalyzer instance.

FINANCE_LEXICON [dict]

Finance words + scores (soared, plunge, downgrade, bullish...) merged into VADER via analyzer.lexicon.update(...).

score(text)

Returns the compound sentiment in [-1, +1].

label(text)

Calls score(); returns 'positive' / 'negative' / 'neutral' using +/-0.05 thresholds.

Key calls / interactions

- analyzer.polarity_scores(text)['compound']
- analyzer.lexicon.update(FINANCE_LEXICON)

Summary

Turns headline text into a number. Extending VADER's lexicon is what lets it read finance language general models miss. Used by runner, seed, fetch_news and batch_test.

finpulse/sentiment/ticker_tagger.py

Responsible for

Ticker tagging - maps a headline to the stock ticker(s) it mentions.

External libraries / modules imported

(none)

Imported from the project

(none)

Functions / key definitions

ALIASES [dict]

keyword -> ticker map (apple->AAPL, tesla->TSLA, ...). The single source of truth for the tracked universe of tickers.

tag(text)

Lowercases + splits text into words; returns the list of tickers whose keyword appears (word-boundary match, so 'apple' != 'pineapple').

Key calls / interactions

- text.lower().split()
- ALIASES.items()

Summary

Connects free text to tickers. ALIASES is reused everywhere (API, dashboard, seed, fetch_news) as the list of tracked stocks - change it in one place and it propagates.

finpulse/storage/db.py

Responsible for

Storage - SQLite persistence of processed headlines.

External libraries / modules imported

sqlite3

Imported from the project

(none)

Functions / key definitions

DB_PATH [const]

'finpulse.db' - the database file.

init_db()

Creates the headlines table if absent (id, text, ticker, score, label, timestamp, url).

insert_headline(text, ticker, score, label, timestamp, url=None)

Inserts one row using parameterized (?) queries.

fetch_all()

Returns all rows ordered by id.

Key calls / interactions

- sqlite3.connect / .execute / .commit / .close

Summary

The pipeline's permanent memory. Parameterized queries avoid SQL injection. Written by runner/seed/fetch_news; read by aggregator, the API and the dashboard.

finpulse/aggregation/aggregator.py

Responsible for

Analytical core - aggregation, price retrieval, sentiment-vs-price alignment, divergence.

External libraries / modules imported

sqlite3, datetime, pandas, yfinance

Imported from the project

finpulse.storage.db.DB_PATH

Functions / key definitions

load_sentiment(ticker)

Reads stored scores into a UTC-indexed DataFrame (pd.read_sql_query).

aggregate_sentiment(ticker, window='1h')

Resamples scores into time windows -> mean + count.

load_price(ticker, period='1mo')

Daily close price Series, index localized to UTC.

align(ticker)

Inner-joins daily sentiment + price on the shared UTC timeline.

detect_divergence(sentiment, price_change_pct, ...)

Pure rule (no I/O): returns a message when sentiment and price point in OPPOSITE directions, else None. A creativity feature, unit-tested.

load_ohlc(ticker, days=30)

Daily OHLC for the last N days (UTC); returns empty safely if yfinance is throttled.

Key calls / interactions

- pd.read_sql_query(...)
- Series.resample(window).agg([...])
- yf.download(...)
- pd.concat([...], join='inner')
- index.tz_localize('UTC')

Summary

Where sentiment meets price. Centralizes the UTC normalization (the project's flagged #1 silent bug). detect_divergence powers the home-page badges. Consumed by the API and dashboard.

finpulse/pipeline/runner.py

Responsible for

Streaming orchestration - a concurrent producer -> queue -> consumer pipeline.

External libraries / modules imported

queue, threading, time, datetime

Imported from the project

news_simulator.get_headlines, analyzer.score, analyzer.label, ticker_tagger.tag, db.init_db, db.insert_headline

Functions / key definitions

q [queue.Queue]

Thread-safe buffer between producer and consumer.

producer()

Puts headlines on the queue, one every 0.5s (simulates a live feed).

consumer()

Pulls headlines, scores + tags them, inserts a row per ticker; stops on a None sentinel.

run()

init_db(), starts both threads, joins the producer, sends the sentinel, joins the consumer.

Key calls / interactions

- get_headlines()
- score() / label() / tag()
- insert_headline(...)
- threading.Thread(...)

Summary

Demonstrates concurrent streaming with decoupled stages that communicate only via the queue and the DB.

Run with: `python -m finpulse.pipeline.runner`.

Responsible for

REST API - a documented, read-only FastAPI service over the stored/aggregated data.

External libraries / modules imported

fastapi.FastAPI, json

Imported from the project

db.fetch_all, aggregator.aggregate_sentiment, aggregator.align, ticker_tagger.ALIASES

Functions / key definitions

app [FastAPI]

The application object; auto-generates Swagger docs at /docs.

to_records(df)

Converts a DataFrame to JSON-safe records (NaN -> null, dates -> ISO).

GET /health

Liveness + stored row count.

GET /sentiment/{ticker}

Daily aggregated sentiment.

GET /alignment/{ticker}

Sentiment aligned against price.

GET /tickers

Tracked tickers (from ALIASES).

GET /headlines?ticker=&limit=

Recent stored headlines (filtered, with URL).

GET /snapshot

Latest sentiment per ticker.

Key calls / interactions

- fetch_all()
- aggregate_sentiment(...)
- align(...)
- sorted(set(ALIASES.values()))

Summary

The programmatic access layer (GET-only by design). Runs with uvicorn finpulse.api.main:app. Parallel to the dashboard - both read the same SQLite store; the dashboard does not call the API.

Responsible for

Dashboard - the 3-page Dash/Plotly UI plus all creativity features.

External libraries / modules imported

time, datetime, urllib.parse, pandas, yfinance, plotly.graph_objects, plotly.subplots, dash (Dash, dcc, html, dash_table, Input, Output)

Imported from the project

db.fetch_all, aggregator.aggregate_sentiment / load_ohlcv / detect_divergence, ticker_tagger.ALIASES, fetch_news.ingest

Functions / key definitions

TICKERS / TIMEFRAMES / PEERS / theme

Config: tracked tickers, lookback windows, peer map, colors.

cached(key, ttl, fn)

Tiny TTL cache so yfinance isn't called too often (avoids rate-limiting).

headlines_df()

Loads all rows from the DB into a timestamp-indexed DataFrame.

snapshot()

Latest daily sentiment per ticker.

latest_price_changes()

Batched yfinance call -> latest price + day % change per ticker.

market_mood() / mood_label() / MOOD_ZONES

Market Mood Index: blends breadth, momentum, stability and sentiment into a 0-100 fear/greed score + its zone text.

build_home_content() / home_page()

Home page: MMI gauge, ticker cards (with divergence badges), top headlines, and the 'Fetch latest news' button.

analytics_page(default_ticker)

Analytics page: ticker + timeframe dropdowns and chart containers.

build_price_comparison(ticker, days)

Normalized %-change line of the ticker vs its peers.

build_news_summary(ticker, ohlc)

Plain-English directional read of recent news (observational).

build_news_feed / news_page / _news_row / _sentiment_pill

News page: filterable card feed.

render / refresh_news / filter_news / update_charts [callbacks]

Routing; fetch-button ingest; news filters; and the main chart callback (sentiment+candlestick, comparison, summary).

Key calls / interactions

- aggregate_sentiment / load_ohlcv / detect_divergence
- ingest()
- yf.download(...)
- plotly make_subplots / Candlestick / Indicator gauge

Summary

The product surface. Reads the DB directly for sentiment and yfinance (cached) for price. Hosts the MMI,

divergence badges, peer comparison, news summary, clickable headlines, timeframe selector, 60s auto-refresh and graceful degradation. Run with `python -m finpulse.dashboard.app`.

seed.py (repo root)

Responsible for

Seed simulated data - generates a year of varied headlines into the DB.

External libraries / modules imported

random, datetime

Imported from the project

analyzer.score / label, ticker_tagger.tag / ALIASES, db.init_db / insert_headline

Functions / key definitions

POSITIVE / NEGATIVE / NEUTRAL [templates]

Headline templates per sentiment class.

seed(days=365)

For each ticker and day, generates 1-2 templated headlines, scores + tags + inserts them; random.seed(42) makes it reproducible.

Key calls / interactions

- init_db()
- score() / label() / tag()
- insert_headline(...)

Summary

Populates the DB with dense synthetic history so the charts have data. Reuses the exact same processing pipeline as the real path.

fetch_news.py (repo root)

Responsible for

Real-news ingestion - pull NewsAPI headlines, score, dedupe, store.

External libraries / modules imported

(none)

Imported from the project

news_api.fetch_headlines, analyzer.score / label, ticker_tagger.ALIASES, db.init_db / insert_headline / fetch_all

Functions / key definitions

ingest(per_company=20)

Per ticker: fetch headlines, skip URLs already stored (dedupe), score + label + insert; returns the count of new rows.

Key calls / interactions

- fetch_headlines(...)
- score() / label()
- insert_headline(...)
- fetch_all()

Summary

The real-data path. Same processing as seed.py but with live headlines. Also imported by the dashboard's 'Fetch latest news' button.

batch_test.py (repo root)

Responsible for

Static test harness - run sentiment + tagging over the sample headlines and print a table.

External libraries / modules imported

(none)

Imported from the project

news_simulator.get_headlines, analyzer.score / label, ticker_tagger.tag

Functions / key definitions

(top-level loop)

Prints score, label and tickers for each of the 20 sample headlines.

Key calls / interactions

- get_headlines()
- tag() / score() / label()

Summary

A quick eyeball test that wires the stages together without storage or streaming - the precursor to runner.py.